

Khulna University of Engineering & Technology

Department : Computer Science and Engineering

Course : CSE-2200 (Advance Programming Laboratory)

Project Title : GhostDocs – The Automated Secure Vault

Date of Submission : November 11, 2026

Submitted By :

Name : Md. Al Intejar Hasnath Prodhan
Roll : 2307094
Lab Group : B2

Submitted to :

1. Mr. Safin Ahmmed Assistant Professor, Dept. of Computer Science and Engineering Khulna University of Engineering & Technology, Khulna	2. Mr. Ehsanul Karim Lecturer, Dept. of Computer Science and Engineering Khulna University of Engineering & Technology, Khulna

Table of Contents

1. Introduction and Purpose 3
1.1 Objectives 3
1.2 Technology Stack 3

2. System Architecture 4
2.1 Android Component Matrix 4
2.2 Processing Pipeline 5

3. Feature Walkthrough 6
3.1 Authentication – Registration and Sign-In 6
3.2 Launch Gate and Biometric Lock 6
3.3 Vault Dashboard 7
3.4 The Invisible Watcher (Foreground Service) 8
3.5 Secure-or-Keep Confirmation Prompt 9
3.6 Smart Crop (OpenCV) 9
3.7 OCR and Smart Classification 10
3.8 AES-256-GCM Encryption 10
3.9 Searchable PDF Generation 11
3.10 Document Actions, Export and Sharing 11
3.11 Air-Gapped QR Share 12
3.12 Import Existing Photos 13
3.13 Settings 13
3.14 User Profile 14
3.15 Home-Screen Widget 15
3.16 Notifications, Permissions and Network Handling 15

4. Object-Oriented Design 16

5. API and Database 17
5.1 External API Summary 17
5.2 Firebase Project Configuration 18
5.3 Firebase Authentication 19
5.4 Cloud Firestore 20
5.5 Firebase Realtime Database 21
5.6 Firebase Storage 22
5.7 Security Considerations for the Back-End 22

6. Testing and Results 23

7. Challenges and Limitations 23

8. Conclusion 24

Appendix A – Screenshot Checklist 25

Appendix B – Source Files Overview 25

1. Introduction and Purpose

Modern smartphones are full of sensitive images: national ID cards, mobile-banking transaction screenshots, handwritten passwords, medical prescriptions and lecture notes. These images normally sit unprotected in the public gallery, where any app with media permission can read them and where they are easily leaked when a phone is lent, lost or backed up to an unsecured location.

GhostDocs is an intelligent Android security application that acts as an invisible background guardian. It continuously watches the device's media store, automatically detects newly captured photos or screenshots, straightens and crops them with computer vision, extracts their text with on-device OCR, classifies them into categories (NID, receipt, prescription, password, study note, other), encrypts them with hardware-backed AES-256-GCM, optionally erases the original from the gallery, and stores the result in a biometric-locked vault. Document metadata is synchronised to Firebase so the vault list is available in real time and survives reinstalls.

1.1 Objectives

- **Protect sensitive images** automatically, without the user having to remember to move them.
- **Keep data private** by running OCR, cropping and encryption entirely on the device.
- **Make documents useful** by producing searchable PDFs and tags from the recognised text.
- **Demonstrate Android components** (Activity, Foreground Service, ContentObserver, BroadcastReceiver, AppWidget, ContentProvider) together with Firebase Authentication, Cloud Firestore and Realtime Database.
- **Apply core OOP principles** (encapsulation, inheritance, polymorphism, abstraction) in a real project.

1.2 Technology Stack

Layer	Technology
Language	Java 17 (no Kotlin)
UI	Native XML layouts (ConstraintLayout / RelativeLayout), Material Components 1.12, RecyclerView, ListView
Build	Android Studio, Gradle, Android Gradle Plugin, Google Services plugin
Authentication	Firebase Authentication (Email/Password and Google Sign-In via play-services-auth 21.2.0)
Cloud database	Cloud Firestore with unlimited persistent offline cache
Realtime telemetry	Firebase Realtime Database (asia-southeast1 instance)
Cloud files	Firebase Storage SDK (declared in build.gradle – see Section 7.5)
OCR	Google ML Kit Text Recognition 16.0.1 (on-device)
Computer vision	OpenCV for Android 4.10.0

Layer	Technology
QR codes	ZXing core 3.5.3
Security	AndroidKeyStore, AES/GCM/NoPadding, AndroidX Biometric 1.1.0
Firebase BOM	com.google.firebase:firebase-bom:35.1.0

2. System Architecture

The application is organised into feature packages under com.ghostdocs.vault: activities, adapter, classifier, model, observer, ocr, receiver, security, service, tasks, util and widget. The project contains roughly 9,900 lines of Java across 37 classes and 13 XML layouts.

2.1 Android Component Matrix

Component	Class	Responsibility
Launcher Activity	LaunchActivity	Splash/auth gate. Checks the Firebase session and shows the BiometricPrompt before opening the vault; otherwise routes to LoginActivity.
Activity (Auth)	LoginActivity	Email/password sign-in and registration with validation (valid e-mail pattern, minimum 6-character password) and Google Sign-In credential exchange.
Activity (Dashboard)	VaultActivity	Main screen: document grid, search box, category chips, document counter, watcher status and watcher toggle button; real-time Firestore listener.
Activity (Settings)	SettingsActivity	ListView with 13 options (profile, theme, auto-start, auto-delete, confirm-before-secure, exclude notes, import, permissions, restore, export PDF, clear vault, about, sign out).
Activity (Profile)	ProfileActivity	Display name update, password change, password-reset e-mail and account/vault data deletion.
Activity (Prompt)	SecurePromptActivity	Asks the user whether a newly captured image should be secured or left in the gallery.
Activity (Import)	ImportPhotosActivity	Gallery picker that retroactively scans and secures photos taken before installation.
Foreground Service	WatcherService	The ‘Invisible Watcher’: START_STICKY, registers the ContentObserver, runs the processing pipeline and publishes status to Realtime Database.
ContentObserver	MediaContentObserver	Listens to MediaStore.Images.Media.EXTERNAL_CONTENT_URI with a 600 ms debounce and duplicate filtering.
BroadcastReceiver	BootReceiver	Restarts the watcher after BOOT_COMPLETED / QUICKBOOT_POWERON when the user enabled auto-start.
AppWidgetProvider	VaultWidgetProvider	Home-screen widget showing document count, watcher ON/OFF and an ‘Open Vault’ shortcut.
ContentProvider	FileProvider	Grants temporary, sandboxed URIs for sharing exported PDFs and JPGs.

Component	Class	Responsibility
Multithreading	VaultExecutor / BaseVaultTask	Four-thread daemon pool executing crop, OCR, encryption and PDF tasks off the main thread.

2.2 Processing Pipeline

When a sensitive photo or screenshot is captured, it flows through six automated stages:

- **Detection –** MediaContentObserver receives the MediaStore change event, debounces it and hands the new image URI to WatcherService.
- **Smart Crop –** CropTask / OpenCvCropper converts the image to grayscale, blurs it, runs Canny edge detection (thresholds 50/150), finds the largest four-corner contour with approxPolyDP and applies a perspective warp so the document appears flat.
- **OCR –** OcrTask runs ML Kit Text Recognition on the cropped bitmap and returns the full text plus bounding boxes (OcrBoundingBox).
- **Classification –** RegexClassifierEngine evaluates a list of DocumentRule implementations (NID, transaction, password, prescription, study note) to assign a document type, a smart title and keyword tags.
- **Encryption –** EncryptTask / AesEncryptionHelper encrypts the image with AES-256-GCM (key in AndroidKeyStore, random 12-byte IV prepended) and, if enabled, deletes the original through ContentResolver.
- **PDF + Sync –** PdfGeneratorTask creates a searchable PDF with an invisible OCR text layer, the DocumentModel is written to Firestore, status is streamed to Realtime Database, the widget is refreshed and a local notification is shown.

3. Feature Walkthrough

This section describes every user-visible feature in the order a new user would encounter it. Each feature is followed by a screenshot placeholder that must be replaced with a real capture from the running app (see Appendix A for a checklist).

3.1 Authentication – Registration and Sign-In

LoginActivity offers two ways to enter the app. In the e-mail flow the user toggles between ‘Sign in’ and ‘Create Account’ modes in the same screen. Input is validated before any network call: the e-mail must match Android’s Patterns.EMAIL_ADDRESS and the password must be at least six characters. Firebase Authentication creates or verifies the account. In the Google flow, GoogleSignInClient launches the account chooser, the returned ID token is exchanged for a Firebase credential and the user is signed in. Errors (wrong password, no network, cancelled chooser) are reported through readable messages.

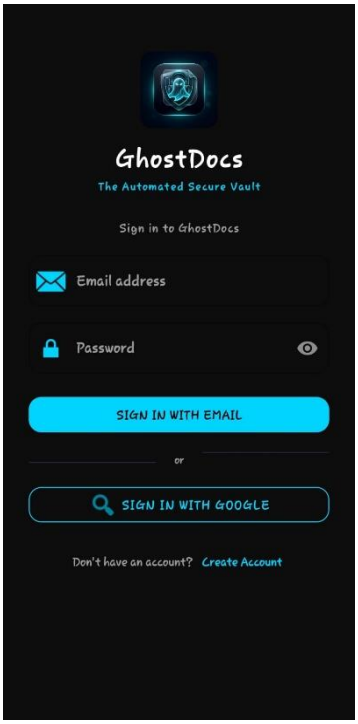


Figure 1: Login / Create Account screen

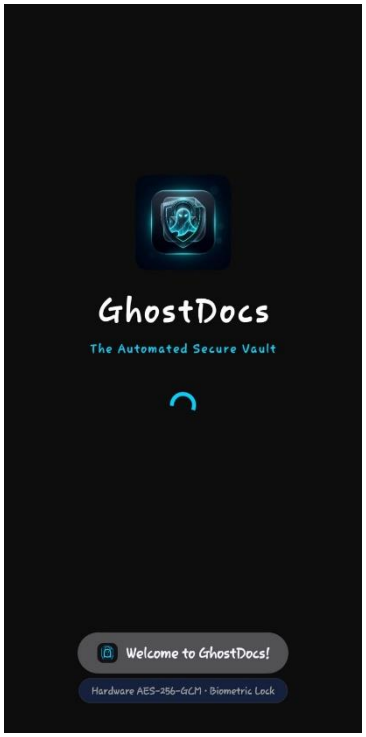


Figure 2: Result after sign-in

3.2 Launch Gate and Biometric Lock

LaunchActivity is the manifest launcher activity. It evaluates FirebaseAuth.getCurrentUser(); if there is no session it opens LoginActivity. If a session exists, BiometricHelper shows a

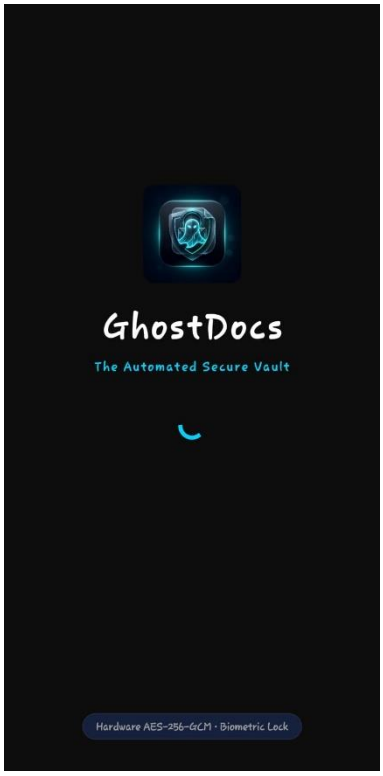


Figure 3: Launch / splash screen

BiometricPrompt titled ‘GhostDocs Vault Lock’ that accepts fingerprint or the device PIN/pattern. The vault opens only after successful authentication, so a stolen unlocked phone still cannot reveal documents. On devices without biometrics the app falls back to the device credential and shows an explanatory message.

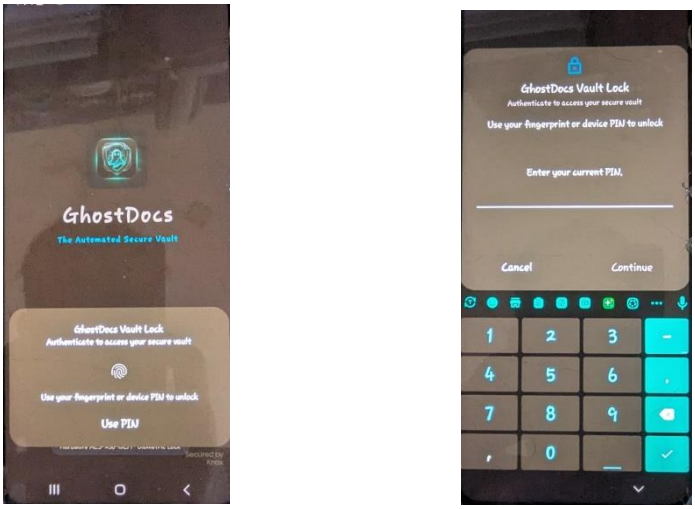


Figure 4: Biometric authentication prompt

3.3 Vault Dashboard

VaultActivity is the heart of the app. It shows a two-column RecyclerView grid of document cards with thumbnail, title, a coloured category badge (colour chosen polymorphically by VaultRecyclerAdapter.resolveBadgeColor()) and date. A header shows the number of secured documents and the current watcher status. A live search box filters by title, OCR text and tags, and horizontally scrolling category chips (All, NID, Receipt, Prescription, Password, Study Note, Other) show per-category counts. The list is fed by a Firestore snapshot listener, so

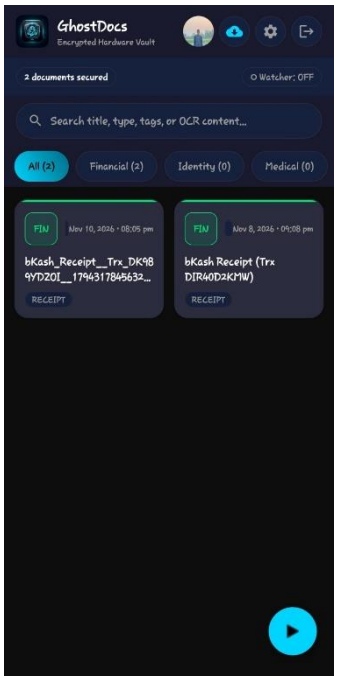


Figure 5: Vault dashboard with documents

changes appear instantly and the offline cache displays documents even without internet. An empty-state view invites the user to import existing photos.

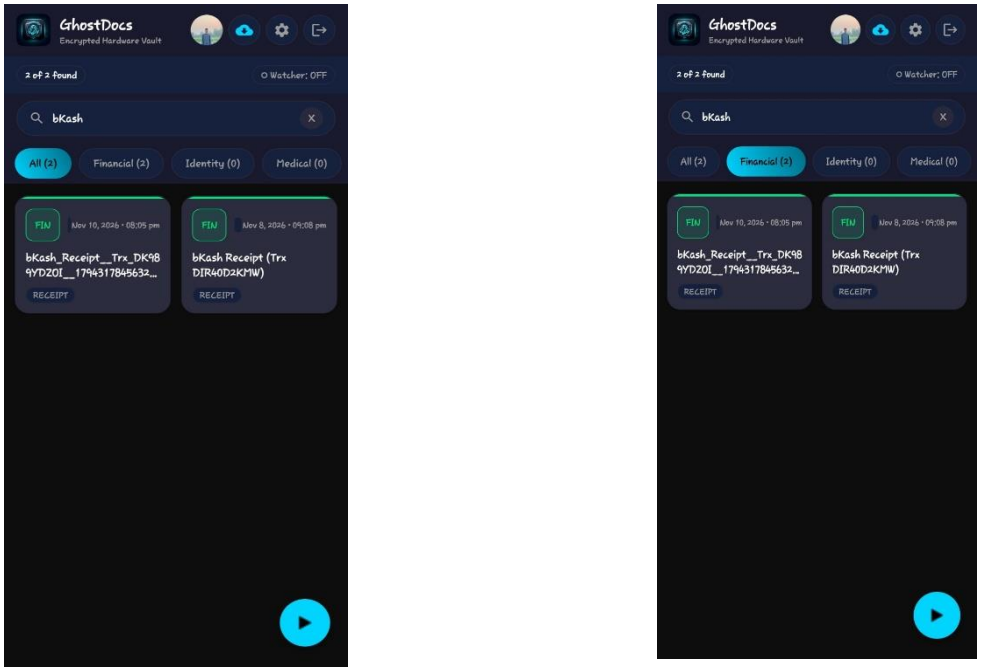


Figure 6: Search / category filter

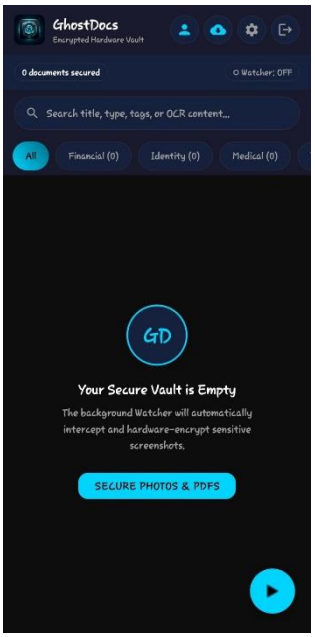
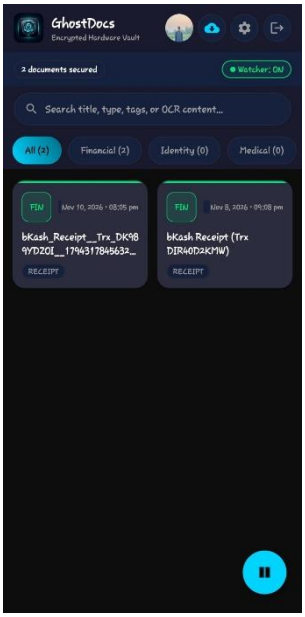


Figure 7: Empty vault state

3.4 The Invisible Watcher (Foreground Service)

A floating action button on the dashboard starts or stops WatcherService. While running, a persistent low-priority notification (‘Vault Watcher Active – Monitoring for sensitive

documents…') is displayed, as required for foreground services (type dataSync on Android 14+). The service registers MediaContentObserver, which waits 600 ms after a MediaStore change to avoid duplicate events and then enqueues processing on VaultExecutor. The service uses START_STICKY and android:stopWithTask="false", and BootReceiver restarts

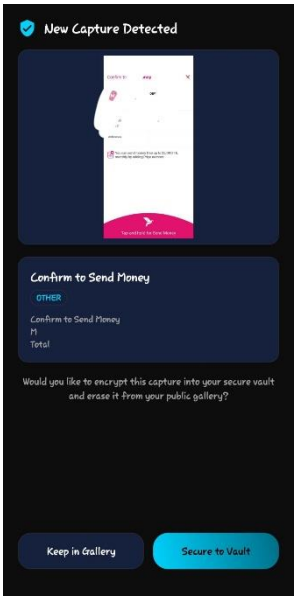


it after reboot if the 'Auto-start on boot' preference is on.

Figure 8: Watcher running – notification and status

3.5 Secure-or-Keep Confirmation Prompt

If the setting 'Confirm before securing' is on, SecurePromptActivity opens when a likely sensitive image is detected. It shows a preview with the suggested smart title and lets the user choose between securing the image in the vault or keeping it in the gallery. If the setting is off ('Auto-Vault'), the pipeline runs silently.



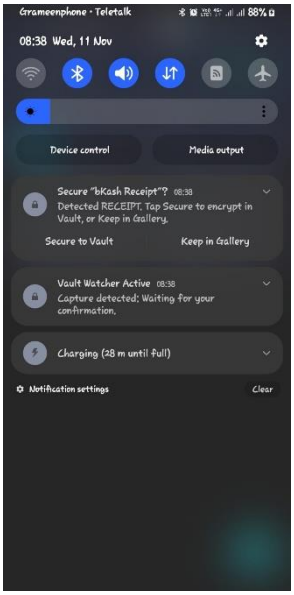


Figure 9: Secure prompt dialog

3.6 Smart Crop (OpenCV)

OpenCvCropper (executed inside CropTask) isolates the document from its background. It converts to grayscale, applies Gaussian blur, runs Canny edge detection, finds contours, approximates them to polygons and picks the largest quadrilateral. The four corners are ordered and a perspective transform (getPerspectiveTransform + warpPerspective) yields a straight, top-down image. Native Mat objects are explicitly released to avoid memory leaks.

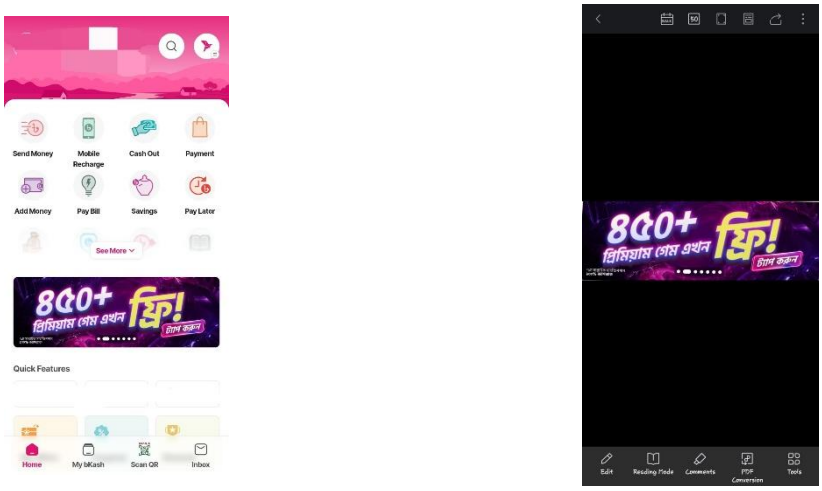


Figure 10: Before / after smart crop

3.7 OCR and Smart Classification

OcrTask uses the bundled ML Kit text recogniser, so no image ever leaves the phone. The text and bounding boxes are passed to RegexClassifierEngine, which holds a list of DocumentRule objects. NidRule looks for ID-card keywords and long digit sequences; TransactionRule finds transaction keywords and TrxID values; PasswordRule finds ‘password’ style labels and values; PrescriptionRule detects Rx markers and extracts doctor and hospital names; StudyNoteRule recognises lecture/notebook pages and subjects. The best match sets docType, a human-readable title and a list of tags (for example ‘TrxID: 9X3AB4KD’).

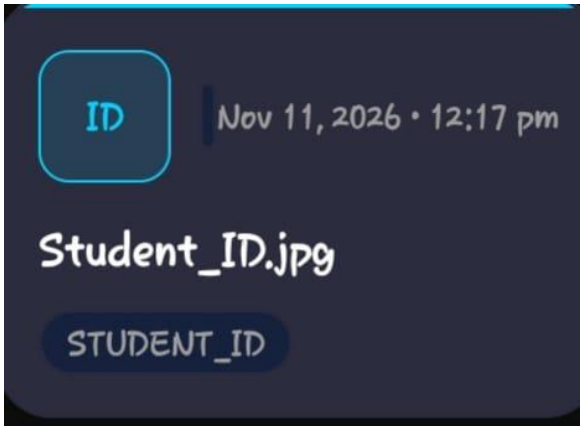


Figure 11: Classified document details

3.8 AES-256-GCM Encryption

AesEncryptionHelper generates a non-exportable AES-256 key inside AndroidKeyStore (alias GhostDocsStorageKey_AES256). Each file is encrypted with AES/GCM/NoPadding using a

fresh 12-byte IV that is stored as the first bytes of the file, followed by ciphertext and a 128-bit authentication tag. Decryption streams the file back into a Bitmap or temporary file only when the user explicitly opens, shares or exports it. If the setting ‘Auto-delete original’ is on, the gallery original is removed with ContentResolver.delete().

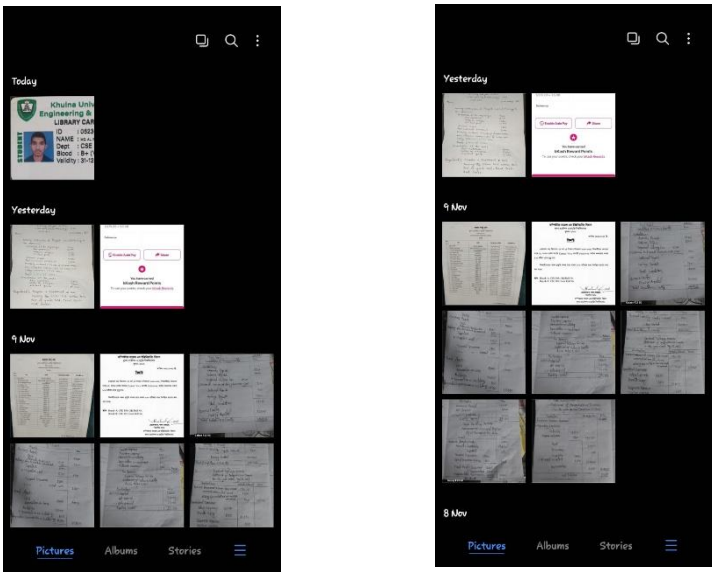


Figure 12: Original removed from gallery

3.9 Searchable PDF Generation

PdfGeneratorTask decrypts the stored image in memory and draws it onto an android.graphics.pdf.PdfDocument page. A fully transparent text layer is drawn over each OCR bounding box so the PDF can be searched and text can be selected in a PDF viewer. The PDF itself is stored encrypted and is decrypted to a temporary file only for export.

3.10 Document Actions, Export and Sharing

Tapping or long-pressing a card opens an action menu: view / export as PDF, export or share as JPG, show QR code, rename, and delete. PDF export offers ‘Save to Downloads’ (Scoped Storage) or ‘Share’. Sharing uses FileProvider so no raw file path is exposed. Rename updates the Firestore title field; delete removes the Firestore document and the local encrypted files after confirmation.

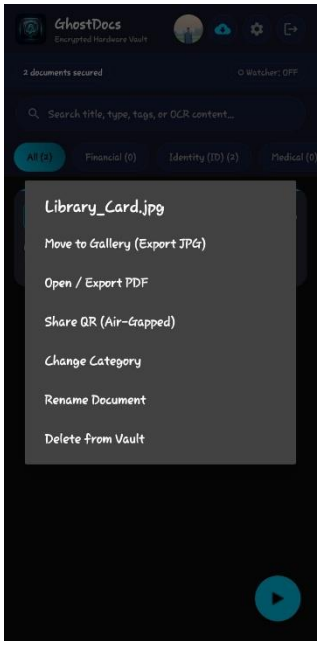


Figure 14: Document action menu

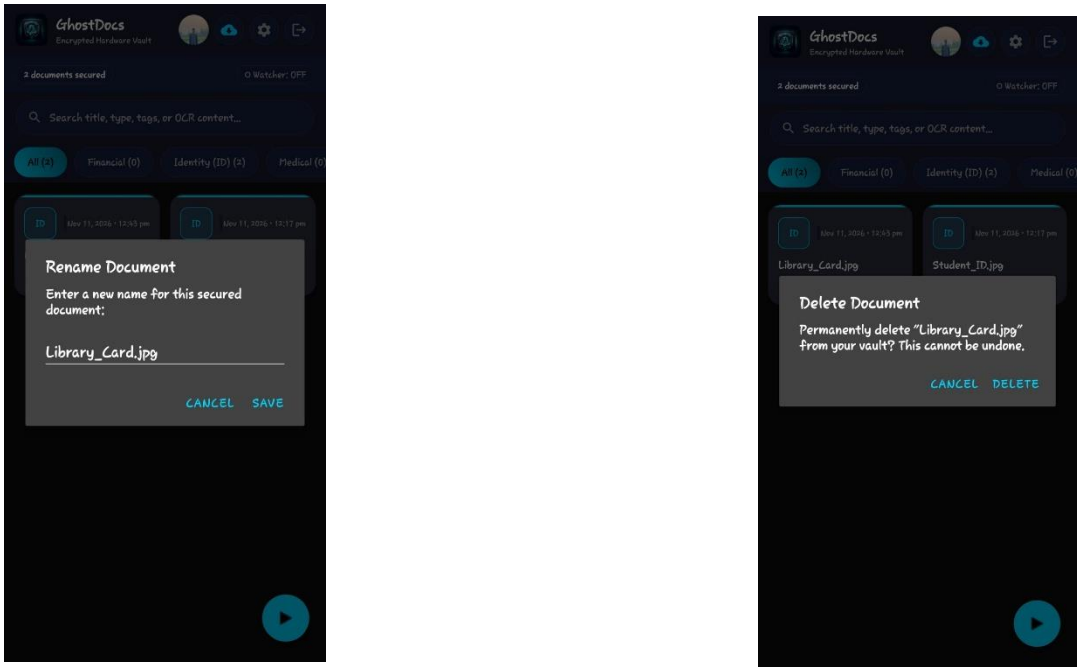


Figure 15: Rename / delete dialog

3.11 Air-Gapped QR Share

QrCodeGenerator uses ZXing core to render a QR bitmap shown in dialog_qr_share. If the document has a share URL it encodes that link; otherwise it encodes a compact summary of the document for offline, air-gapped transfer. The user can scan it with another phone without any network connection.



Figure 16: QR share dialog

3.12 Import Existing Photos

ImportPhotosActivity addresses the fact that old documents were taken before GhostDocs was installed. It queries MediaStore, shows a selectable gallery grid (GalleryPhotoAdapter with background-loaded thumbnails), and runs the same crop → OCR → classify → encrypt → PDF → Firestore pipeline on the selected images with progress feedback.

3.13 Settings

SettingsActivity presents a ListView using the custom LogListAdapter. Options: Profile; App Theme (Dark / Light / Follow system, applied by ThemeHelper); Auto-start watcher on boot; Auto-delete original after encryption; Confirm before securing; Exclude study notes (keep them in the gallery); Import old photos; Permissions; Restore all photos to gallery (decrypts back to MediaStore); Export vault as PDF; Clear vault (batch-deletes all Firestore documents); About; Sign out.

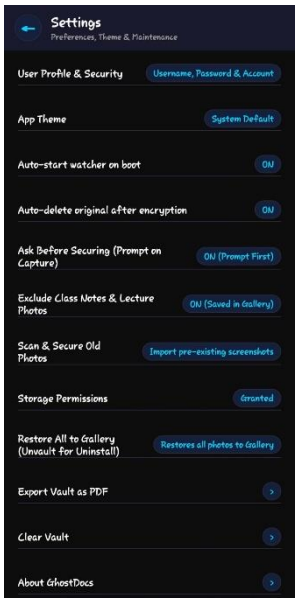


Figure 19: Settings screen

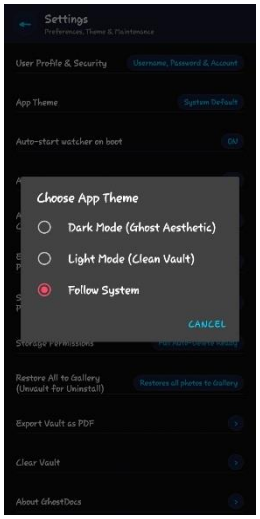


Figure 20: Theme selection / dark mode

3.14 User Profile

ProfileActivity lets the user change the display name (FirebaseUser.updateProfile), change the password (FirebaseUser.updatePassword, with strength validation and re-authentication when required), send a password-reset e-mail (sendPasswordResetEmail) and delete the account data (a Firestore batch deletes users/{uid}/documents and the user document).



Figure 21: Profile screen

3.15 Home-Screen Widget

VaultWidgetProvider renders widget_vault.xml with the live document count, the text '● Watcher: ON' or '○ Watcher: OFF', and an 'Open Vault' button. It updates on APPWIDGET_UPDATE and whenever the pipeline secures a new document.



Figure 22: Home-screen widget

3.16 Notifications, Permissions and Network Handling

NotificationHelper creates two channels: a low-importance channel for the persistent watcher notification and a high-importance ‘Document Alerts’ channel that announces ‘Secure Document Detected – Encrypted and stored in your vault’. PermissionHelper requests READ_MEDIA_IMAGES (Android 13+) or READ_EXTERNAL_STORAGE and POST_NOTIFICATIONS at runtime. NetworkHelper detects Airplane Mode and loss of connectivity and shows context-specific dialogs, protecting cloud synchronisation from hanging.



Figure 23: Document alert notification

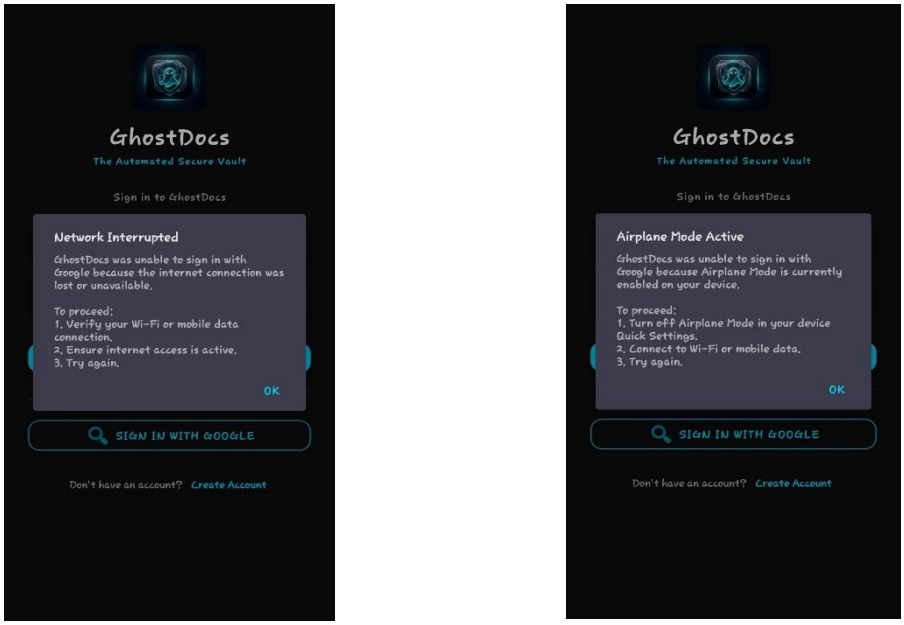


Figure 24: Permission / network dialog

4. Object-Oriented Design

Principle	Where it is applied
Encapsulation	DocumentModel keeps private fields behind getters/setters and serialises itself to org.json.JSONObject. AesEncryptionHelper hides the keystore alias, cipher transformation and IV handling. OpenCvCropper hides Mat allocation and release.
Inheritance	CropTask, OcrTask, EncryptTask and PdfGeneratorTask extend BaseVaultTask<T>. VaultRecyclerAdapter extends RecyclerView.Adapter. LogListAdapter extends ArrayAdapter.
Polymorphism	The DocumentRule interface is implemented by NidRule, TransactionRule, PasswordRule, PrescriptionRule and StudyNoteRule; the engine treats them uniformly. Each task overrides doInBackground(). Badge colours are resolved per document type.
Abstraction	BiometricHelper callback contract, TaskCallback<T> for asynchronous results, and the DocumentRule interface let callers depend on behaviour, not implementation.

5. API and Database

GhostDocs does not call a custom REST service. Its external 'APIs' are Google/Firebase service APIs reached through official SDKs, plus on-device library APIs. This section documents each of them and the Firebase back-end configuration, with screenshots to be captured from the Firebase Console and from Logcat.

5.1 External API Summary

API / SDK	Used by	Purpose and response
Firebase Authentication API	LoginActivity, ProfileActivity, LaunchActivity	Creates/verifies users and returns a FirebaseUser (UID, e-mail, display name, ID token).
Google Sign-In API (play-services-auth)	LoginActivity	Shows the Google account chooser and returns GoogleSignInAccount with an ID token that is exchanged for a Firebase credential.
Cloud Firestore API	VaultActivity, WatcherService, ImportPhotosActivity, SettingsActivity, ProfileActivity	Reads/writes DocumentModel metadata; snapshot listener returns live QuerySnapshot results.
Realtime Database API	WatcherService	Publishes watcher status and recent captures; onDisconnect marks the device OFFLINE.
ML Kit Text Recognition API	OcrTask	Returns a Text object with blocks, lines, elements and bounding boxes (on-device).
ZXing / OpenCV libraries	QrCodeGenerator, OpenCvCropper	Local computation (QR BitMatrix, edge/contour/perspective results).

Screenshots of external API responses

5.2 Firebase Project Configuration

The app is registered in the Firebase project ghostdocs-57c78 with package name com.ghostdocs.vault (and a second client for com.ghostdocs.vault.debug, matching the debug build's applicationId suffix). The google-services.json file placed in the app/ directory is processed by the com.google.gms.google-services Gradle plugin. Key values are:

Setting	Value
Project ID	ghostdocs-57c78
Project number	535966032522
Realtime DB URL	https://ghostdocs-57c78-default-rtdb.asia-southeast1.firebasedatabase.app
Storage bucket	ghostdocs-57c78.firebasestorage.app
Android package	com.ghostdocs.vault (+ .debug)
SHA-1 registered	Debug certificate fingerprint (required for Google Sign-In)

5.3 Firebase Authentication

Two sign-in providers are enabled: Email/Password and Google. Every vault document is tied to the UID of the signed-in user, which is also used as the key for Firestore sub-collections and Realtime Database nodes.

5.4 Cloud Firestore

Firestore stores the structured metadata for each secured document. The data model follows the hierarchy users/{uid}/documents/{docId}, so each user can only ever query their own collection. Offline persistence is enabled in GhostDocsApp with PersistentCacheSettings and CACHE_SIZE_UNLIMITED, allowing the dashboard to render from the local cache instantly and sync in the background.

Field	Type	Description
docId	string	Unique ID of the document (also the Firestore document ID).
title	string	Smart title generated by the classifier (editable via Rename).
ocrText	string	Full text recognised by ML Kit; used by the search box.
encryptedPath	string	On-device path of the AES-256-GCM encrypted image.
thumbnailPath	string	Reference used to render the card thumbnail.
pdfPath	string	Path of the generated searchable PDF.
timestamp	number	Capture/secure time in milliseconds.
tags	array<string>	Extracted keywords, e.g. ‘TrxID: …’, ‘NID: …’.
docType	string	NID, RECEIPT, PRESCRIPTION, PASSWORD, STUDY_NOTE or OTHER.
ownerId	string	UID of the owning user.
shareUrl	string	Optional cloud link used for QR sharing (empty by default).

Representative code (WatcherService) that writes a document:

```
FirebaseFirestore.getInstance()
    .collection("users").document(currentUser.getUid())
    .collection("documents").document(docId)
    .set(docModel)
    .addOnSuccessListener(unused -> Log.d(TAG, "Metadata pushed to Firestore: " +
docId))
    .addOnFailureListener(e -> Log.e(TAG, "Failed pushing metadata to Firestore",
e));
```

5.5 Firebase Realtime Database

Realtime Database provides low-latency telemetry about the background watcher. WatcherService writes to watcher_status/{uid} every time its state changes (ACTIVE, SCANNING, PROCESSING, STOPPED, OFFLINE). A server-side onDisconnect() handler automatically sets the status to OFFLINE with a lastSeen timestamp if the device loses

connection or the service is killed. Each secured capture also pushes an entry under recent_captures.

```
watcher_status
  └── {uid}
        ├── status      : "ACTIVE" | "SCANNING" | "PROCESSING" | "STOPPED" |
"OFFLINE"
        ├── details     : "Secured: <title>"
        ├── device      : "<manufacturer> <model> (Android <version>)"
        ├── updatedAt   : <server timestamp>
        ├── lastSeen    : <server timestamp, set by onDisconnect>
        └── recent captures
              └── {pushId} : { displayName, docType, title, tags[], detectedAt }
```

5.6 Firebase Storage

The Firebase Storage SDK (com.google.firebase:firebase-storage) is declared in app/build.gradle and the bucket ghostdocs-57c78.firebasestorage.app is configured in google-services.json, intended for the QR ‘cloud view link’ feature. In the submitted source code, however, no class uploads files to Firebase Storage: document images and PDFs are deliberately kept only on the device in encrypted form, and the Firestore shareUrl field is empty unless a link is supplied. Because the assignment says Storage is required only ‘if applicable’, this report treats it as available but not exercised. If the instructor requires it, a screenshot of the (empty or populated) Storage > Files tab can be added below.

5.7 Security Considerations for the Back-End

- **Per-user isolation:** all data is placed under the signed-in user’s UID; Firestore rules should restrict reads and writes to request.auth.uid == uid.
- **Sensitive content stays local:** images and PDFs are encrypted on the device; only metadata and OCR text are synchronised. Because ocrText can itself contain sensitive text (such as an ID number), the Firestore rules and project access must be locked down.
- **Key protection:** the AES key is generated in AndroidKeyStore and cannot be exported.
- **Backup disabled:** android:allowBackup="false" and android:hasFragileUserData="true" prevent cloud backup and preserve data prompts on uninstall.

6. Testing and Results

#	Test case	Expected result	Result
1	Register with an invalid e-mail	Inline validation error, no Firebase call	[Pass / Fail]
2	Register with password under 6 characters	Validation error shown	[Pass / Fail]
3	Sign in with Google	Account chooser, vault opens after biometric	[Pass / Fail]

#	Test case	Expected result	Result
4	Start watcher and take a screenshot of an ID card	Notification, new NID document in vault, original deleted	[Pass / Fail]
5	Take a bKash-style transaction screenshot	Classified as RECEIPT with TrxID tag	[Pass / Fail]
6	Search for a word from a document's OCR text	Matching cards only	[Pass / Fail]
7	Turn on Airplane Mode while watcher is running	Network dialog; RTDB status becomes OFFLINE; documents still visible offline	[Pass / Fail]
8	Reboot the phone with auto-start on	Watcher restarts automatically	[Pass / Fail]
9	Export a document as PDF and search its text	PDF opens and text is searchable	[Pass / Fail]
10	Clear vault	All Firestore documents removed	[Pass / Fail]

Note: the Result column must be filled in by the student after running each test on a physical device.

7. Challenges and Limitations

- **Foreground-service restrictions:** Android 14 requires a declared foreground service type (dataSync) and POST_NOTIFICATIONS runtime permission; the service and permission helper handle both.
- **Scoped Storage:** deleting gallery originals requires careful use of ContentResolver and, on some versions, user consent.
- **Duplicate MediaStore events:** a single capture fires several change events; solved with a 600 ms debounce and duplicate filtering.
- **Heuristic classification:** regular expressions are fast and private but can misclassify unusual layouts; a trained model could improve accuracy.
- **Metadata sensitivity:** OCR text is stored in Firestore; strict security rules or client-side encryption of that field is recommended.
- **Storage not exercised:** cloud file sharing through Firebase Storage is not implemented in the current code.

8. Conclusion

GhostDocs demonstrates how a set of Android platform components, Firebase services and on-device AI libraries can be combined into a practical privacy tool. It automatically finds sensitive images, converts them into encrypted, classified and searchable documents, protects access with biometrics, and keeps a live cloud index through Firestore while reporting watcher health through Realtime Database. The project applies encapsulation, inheritance, polymorphism and abstraction consistently, and runs all heavy work on a dedicated thread

pool. Future work includes Firebase Storage-based share links, client-side encryption of metadata, a machine-learning document classifier, and multi-device vault synchronisation.

Appendix A – Screenshot Checklist

Replace every dashed placeholder box with a real screenshot before printing. Suggested capture list:

- **App:** Login, Google chooser, biometric prompt, vault with documents, empty state, search/filter, watcher notification, secure prompt, crop before/after, document actions, QR dialog, import screen, settings, theme dark/light, profile, widget, alert notification.
- **Firebase Console:** Project settings, Authentication (providers and users), Firestore data and rules, Realtime Database data (ACTIVE and OFFLINE), Storage (optional).
- **Android Studio Logcat:** Google sign-in result, OCR output, Firestore and Realtime DB success messages.

Appendix B – Source Files Overview

Package	Classes
activities	LaunchActivity, LoginActivity, VaultActivity, SettingsActivity, ProfileActivity, SecurePromptActivity, ImportPhotosActivity
service / observer / receiver / widget	WatcherService, MediaContentObserver, BootReceiver, VaultWidgetProvider
tasks	BaseVaultTask, VaultExecutor, CropTask, OcrTask, EncryptTask, PdfGeneratorTask
classifier	DocumentRule, RegexClassifierEngine
security	AesEncryptionHelper, BiometricHelper
util	OpenCvCropper, QrCodeGenerator, PdfExportHelper, NotificationHelper, PermissionHelper, NetworkHelper, ThemeHelper
model / adapter / ocr	DocumentModel, GalleryPhotoItem, CategoryChipModel, VaultRecyclerAdapter, GalleryPhotoAdapter, CategoryChipAdapter, LogListAdapter, OcrBoundingBox